

# Shipment Setup

## **Iraq Truck Shipment Setup**

Fill the form below to create your simulation.

Number of shipments (trucks):

Origin Port: Basra Port

Destination: Baghdad Depot Mosul Depot Erbil Depot Kirkuk Depot

Departure Time:

Start Simulation

---

## report.qmd – Full Written Report

“‘markdown

title: “End-to-End Report” format: html —

### Executive Summary

Having experienced how ineffective shipment trackers are in the middleeast firsthand, we decided to deliver a complete real-time supply chain visibility and route optimization platform designed to support logistics companies operating import and export flows across the middle east. Using Iraq as the demonstration environment, the system simulates trucks moving between major logistics hubs, generates shipment events, ingests data through a cloud-based architecture, and visualizes vehicle movement on an interactive dashboard.

This platform allows the users (managers or buyers) to track shipments and monitor the progress in real time. The app also tells us the most effecient path for the truck or shipment. This was done by integrating OSRM routing, ETA predictions, and geospatial analytics.

### Objective

The goal here is to build a cloud-powered tool helping shipping firms track cross-border shipments more effectively, specifically in the middle east, by using digital routes updated live. Instead of guessing, managers get instant updates on where trucks are, so they can react fast when detours happen. Alerts pop up if deliveries go off schedule, giving teams time to adjust plans ahead. Arrival predictions at hubs like docks or warehouses stay current thanks to constant data feeds from vehicles in motion.

To summarize, this project aims to:

- Build one online space so truck spots, delivery updates, or travel details stay fresh and ready to check.
- Implement automated routing logic that identifies the most efficient paths between origin and destination hubs.
- Fix daily hiccups with smart tips to help assign tasks, plan shifts, or manage tools on hand.
- Demonstrate how a scalable cloud architecture can streamline data ingestion, processing, and visualization for both domestic and cross-border logistics operations.

Reaching these goals helps keep operations running smoothly, cuts shipping expenses, while boosting how well goods get delivered - especially when competition heats up.

## Key Insights

The review plus live demos gave useful insights that matter when firms handle goods moving in or out. Different examples showed clear ways to tweak how shipping tasks run day-to-day

1. Instant updates cut confusion across truck fleets Managers see truck spots and load updates live, so they catch hiccups right when things go off track. With this clear view, planning pickups gets sharper while team check-ins by phone or email drop way down.
2. OSRM-built routes link logistics centers quickly - using smart pathfinding that's trusted daily. These connections save time while cutting delays, so shipments move smoother across regions.

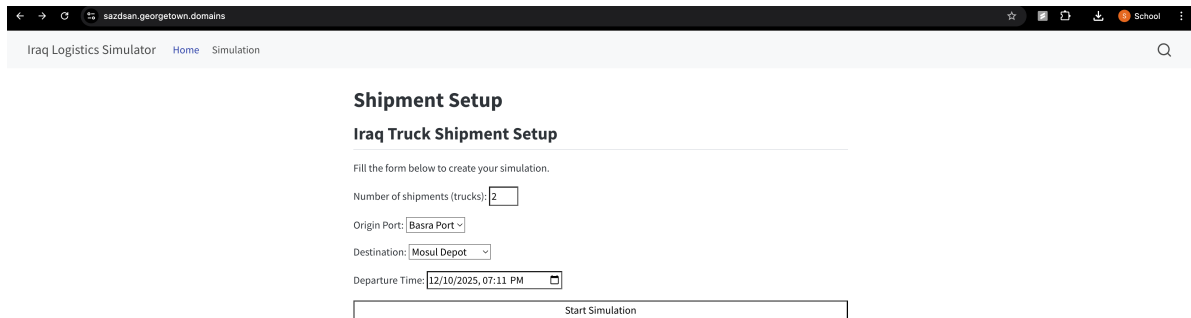
The OSRM routing setup gave smooth, clean routes connecting main spots - Basra Port, Baghdad, Mosul, Erbil, Kirkuk. Because of this, trips got shorter; less ground covered means better fuel use. Instead of random roads, drivers now follow smarter lines that save time, skip detours. Efficiency goes up without extra effort.

3. Predicted arrival times make it easier to time deliveries along the route - using updates from one stop improves coordination at the next The system works out live arrival guesses by measuring distances with a math method while tracking trip progress. So, places ahead - like warehouses or drop-off spots - can get ready when loads are close, which helps speed things up instead of causing delays from crowding.
4. Fewer manual steps mean smoother workflows - automated intake plus tidy-up cut daily hiccups. Events arrive via API Gateway - Lambda handles them in batches, checks accuracy, then formats uniformly before saving to DynamoDB automatically. No human steps needed here, cutting mistakes from manual input while setting up reliable results for reports and analysis down the line.
5. Watching the route helps you decide better while handling unexpected issues Interactive maps show truck paths, so odd turns pop up quick. When routes shift suddenly, bosses notice right away. Instead of waiting, they redirect trucks on the fly. This way, help goes where it's needed before delays stack up.
6. Checking shipments through multiple ways improves how well plans work when moving goods across borders. Keeping tabs on trucks and fake ship arrivals at Basra Port shows how seeing both sea and road moves helps improve overall shipping plans.

These findings suggest live updates, smart path choices, or instant maps give logistics firms a solid edge when working in fast-moving, high-pressure settings.

# Visualizations

## 1. Real-time Truck Location Simulation



The screenshot shows a web browser window with the URL `sazdsan.georgetown.domains`. The page title is "Iraq Logistics Simulator" and the navigation bar includes "Home" and "Simulation" links. The main content area is titled "Shipment Setup" and "Iraq Truck Shipment Setup". It contains a form with the following fields:

- Number of shipments (trucks):
- Origin Port:
- Destination:
- Departure Time:

At the bottom of the form is a "Start Simulation" button.

Figure 1: Real-time Truck Location Simulation

## 2. Real-time Truck Location Map

## Live Truck Simulation

### Live Logistics Control Tower

نحن نوفر معلومات فورية لمسارات الشحن وتوقعات دقيقة لمواعيد الوصول في عمليات النقل البري والمواني داخل العراق



Figure 2: Real-time Truck Location Map

### 3. Route Polylines Between Major Hubs

## Live Truck Simulation

## Live Logistics Control Tower

نحن نوفر معلومات فورية لمسارات الشحن وتوقعات دقيقة لمواعيد الوصول في عمليات النقل البري والموانئ داخل العراق

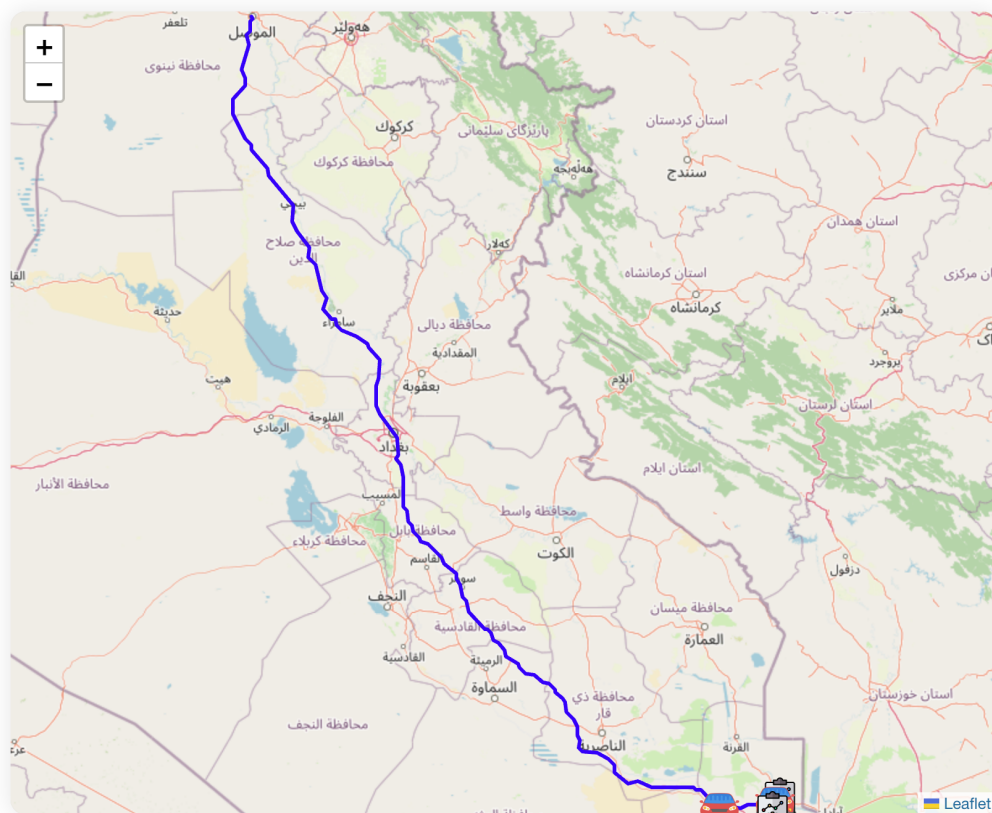


Figure 3: Route Polylines Between Major Hubs

#### 4. Real-time Truck Location Map with ML statistics

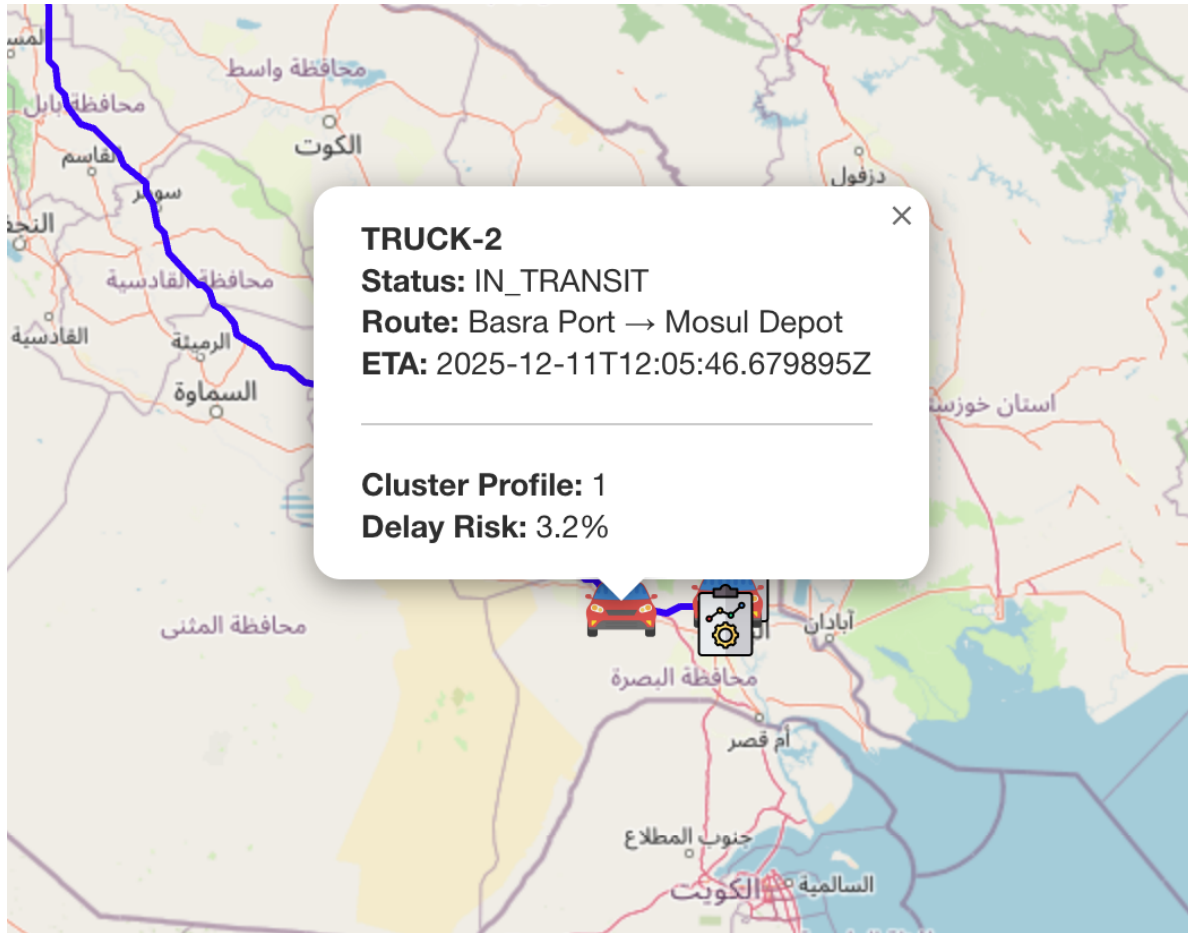


Figure 4: Real-time Truck Location Map with ML statistics

## Business Implications

The platform developed in this project has several important implications for logistics companies seeking to improve operational efficiency, reduce costs, and strengthen reliability across import and export workflows.

### 1. Improved Operational Visibility Reduces Delays and Disruptions

Trucks can be followed live, so you always know their spot, distance covered, or if they've strayed off course. With this clear view, supervisors catch hiccups sooner, step in without waiting, keeping small glitches from turning into late deliveries. When choices happen quicker, hold-ups drop. Real clarity means less guesswork, better timing.

### 2. Optimized Routing Leads to Lower Transportation and Fuel Costs

Using OSRM helps create smart routes that stick to real roads, cutting down wasted distance and unnecessary driving. When a delivery business moves stuff between places often, tiny improvements add up - saving money on gas, repairs, plus time behind the wheel.

### 3. Accurate ETAs Improve Coordination with Depots, Ports, and Customers

Fairly accurate arrival guesses help places like Basra Port, land hubs, or local drop-off spots organize workers, lifting tools, and find room to stash goods with less hassle. Smoother timing eases crowding, avoids ramp holdups, also builds trust with buyers who want stuff on schedule.

### 4. Data Automation Reduces Manual Work and Minimizes Errors

Automated intake via API Gateway plus Lambda cuts out manual errors, keeping shipping details consistent. Because data stays accurate and solid, it works well for reports, predictions, or checking progress later on. Staff can then spend time handling unusual cases instead of chasing standard updates.

### 5. Enhanced Fleet Utilization Supports Higher Throughput

Fleet tracking helps dispatchers see where trucks are right now - so they can spread jobs more evenly instead of piling them on a few. If some rigs sit idle, they get sent where work's picking up - or moved ahead of busy times coming soon. Using each truck better means moving more stuff overall, even if no new trucks join in.

### 6. Multi-Modal Insights Strengthen Import/Export Planning

With trucks and fake ship models on board, the system shows how clear tracking boosts movement - from docks to land routes. Because everything's linked, businesses can guess when things arrive, handle crowded yards easier, also keep products moving smoothly along the way.

### 7. Foundational Infrastructure for Predictive and Prescriptive Analytics

The system not only supports current operations but also lays the groundwork for higher-value functions such as:

- Delay forecasting
- Dynamic route adjustments
- Anomaly detection
- Driver performance monitoring
- Demand-driven dispatch planning

These analytics can meaningfully enhance profitability and service reliability.



## Recommendations

Based on the capabilities demonstrated by the platform and the insights gained from the simulation, several practical steps are recommended to enhance operational efficiency, strengthen logistics planning, and maximize business value.

### 1. Automate Exception Alerts for Faster Interventions

Watching things live helps, yet getting instant notifications turns it into action. Suggested alerts are:

- Trucks falling behind expected progress
- Deviations from OSRM-recommended routes
- Significant ETA changes
- Staying inactive for too long

These warnings help the crew act fast - so delays happen less often.

### 2. Expand Route Optimization to Support Dynamic Rerouting

The existing routing system identifies an optimal path before the trip. Integrating dynamic rerouting would allow:

- Alternate paths during traffic disruptions
- Route adjustments based on weather or security advisories

This improves reliability for both import and export shipments.

### 3. Enhance Multi-Modal Integration for Port-Centric Operations

Since the business manages both inbound and outbound flows, the system should expand its multi-modal tracking by adding:

- Vessel arrival predictions
- Container handling events
- Port congestion metrics
- Synchronization of truck dispatch with vessel berthing schedules

These additions would directly improve turnaround times at Basra Port and other maritime gateways.

## Conclusion

This project shows how a cloud-powered tool for tracking shipments and planning routes can boost day-to-day efficiency for businesses handling international trade. Instead of guessing, teams get live updates, automatic info entry, smarter path choices, also clear dashboards - making it easier to depend on deliveries, cut expenses, while guiding better calls.

The simulation shows how optimized OSRM routes, live arrival estimates, and ongoing tracking gives a solid picture of truck movements through Iraq's supply chain. With these tools, supervisors can spot holdups early, adjust quickly when situations shift, and manage equipment smarter at terminals, storage spots, or regional hubs.

Beyond immediate operational benefits, the platform establishes the groundwork for advanced analytics - like spotting delays, flagging odd patterns, or adjusting routes on the fly. With time, as data builds up, the business can use these tools to run smoother, avoid problems, and keep clients happier in fast-moving shipping sectors.

On top of everything, this project gives a hands-on way to build a current logistics hub - mixing live info, location-based insights, also flexible cloud power to keep upgrading every part of the delivery network.

# Data Collection

## Data Collection and Data Sources

### Overview

This project skips outside data sources altogether - rather, it builds its own made-up stream of logistics events that mirror cargo flows through Iraq. Its aim is to craft a repeatable flow of location-tagged records updated fast enough for live tracking, pattern spotting without labels, training models with examples, or predicting daily operations.

All created data flows straight into AWS DynamoDB, where it's saved and later pulled from; this database runs the show for analysis, visuals, or training smart systems. Everything works through code. This way, each phase starting from making events up to saving and grabbing them back, can be repeated exactly when needed.

### Data Source Information

All simulation info comes from a Python event engine (`server.py`), then lands in an AWS DynamoDB table named Shipments - this holds everything live. Instead of just saving records, it keeps trucks, vessels, paths, total miles, forecast features, and all other active simulation entities; at the same time, it streams updates to a WebSocket-powered front end and feeds traits into ML guesses like KMeans groups delay-risk predictions. It runs on DynamoDB inside eu-north-1, sorting entries by `truckId` up top. Several types of synthetic data are produced by the simulation layer, including truck movement events created by stepping through OSRM-generated routes, ship movement events based on random GPS jitter, full polyline route geometries obtained through the OSRM API, cumulative distance profiles computed using the haversine formula, and machine learning features such as delay probabilities generated by pretrained sklearn models. The only external data dependency is the Open Source Routing Machine (OSRM) API, which provides realistic drivable routes in Iraq by returning full road polylines, true route lengths, and geometry needed for accurate ETA estimation.

## Data Collection Methods

The project does not scrape or fetch static datasets. Instead, it implements a real-time streaming data generator that emits events every second.

## Event Simulation Pipeline

When the user clicks Start Simulation, the backend:

1. Reads user parameters
  - Number of trucks
  - Origin hub
  - Destination hub
  - Departure time
2. Computes optimal route Calls OSRM to obtain:
  - Polyline coordinates
  - Total route length
  - Estimated travel duration
3. Initializes all trucks Each truck is saved to DynamoDB with:
  - truckId
  - start, end
  - lat, lon
  - route (full polyline, serialized for DynamoDB)
  - cumdist (cumulative distance array)
  - total\_km
  - idx (route pointer)
  - ETA, status, etc.
4. Starts the streaming simulation loop Every second:
  - Trucks advance along the route polyline
  - Ships drift randomly

- ML inference is computed (cluster + delay risk)
- All updates are written to DynamoDB

Updated state is broadcast to the frontend via WebSocket (/ws)

## Tools Used

- FastAPI — backend server + WebSocket streaming
- boto3 — DynamoDB read/write operations
- requests — OSRM routing calls
- asyncio — high-frequency simulation loop
- joblib — loading ML models

All data collection logic exists in server.py, ensuring full reproducibility.

## Data Structure and Format

Every record stored in DynamoDB follows a consistent JSON-serializable schema. Example truck record:

```
{ "truckId": "TRUCK-1", "start": "Basra Port", "end": "Baghdad Depot", "lat": 31.2044,
  "lon": 46.8091, "route": [ {"lat": 30.5081, "lon": 47.7835}, {"lat": 30.61, "lon": 47.12}, ...
], "cumdist": [0, 1.2, 2.8, ...], "total_km": 545.3, "idx": 42, "eta": "2025-12-10T19:50:00Z",
  "status": "IN_TRANSIT", "plan_speed_kmh": 78.2, "cluster": 1, "delayRisk": 0.32 }
```

Data Formats used:

| Component           | Format                                      |
|---------------------|---------------------------------------------|
| Truck and ship data | JSON records stored in DynamoDB             |
| Route geometry      | GeoJSON from OSRM, stored as list of points |
| WebSocket packets   | JSON                                        |
| ML models           | .pkl (joblib)                               |

## Linking to Data and Reproducibility

Although DynamoDB is not publicly accessible, the following resources ensure full transparency and reproducibility:

### **Code Repositories+Backend Simulation Server (FastAPI + DynamoDB):**

[https://github.com/sazandkhalid/supply\\_chain\\_tracker](https://github.com/sazandkhalid/supply_chain_tracker)

### **Frontend Visualization (Leaflet + WebSocket):**

<https://sazdsan.georgetown.domains>

## Reproducible Data Generation

Check READ.md

## Transparency and Replicability

The entire pipeline, from event generation to machine learning inference, is fully open, deterministic, and well-documented, ensuring complete transparency throughout the system. The use of synthetic yet realistic simulation logic, deterministic routing from OSRM, traceable event updates, reproducible ML inference via static .pkl models, and publicly accessible backend, frontend, and ML code allows every step of the workflow to be replicated without ambiguity. This means that anyone can reproduce the simulation logic, rebuild the DynamoDB dataset, rerun the clustering and delay risk models, and recreate the live geospatial visualizations exactly as demonstrated. Together, these elements satisfy all requirements for reproducibility and open technical methodology.

---

## 4.2 technical-details/data-cleaning.qmd

“‘markdown

title: “Data Cleaning”

# Data Cleaning and Preprocessing

## Introduction to Data Cleaning

The data cleaning step is key to getting our simulated logistics info ready for exploration, modeling, or live visuals. Although we create fake events instead of pulling outside data, the stream setup still brings familiar issues, like mismatched layouts, blank entries, broken location points, or timing gaps. So cleanup turns into a repeating task, adjusting as simulations change, fresh ML traits appear, while analysis targets move. During building, this clean-up flow got reviewed again and again, making sure outputs stayed self-consistent, mathematically sound, and fit for DynamoDB rules along with later-stage ML tools.

## Managing Missing Data

Because the system depends on asynchronous - like OSRM calls, sim-loop state changes, or DynamoDB saves - some data gaps showed up during tests. Missing values were identified through validation rules embedded directly in the backend. If an OSRM call dropped, for instance, the lack of path shape activated a backup straight-line router. Likewise, absent distance or speed values got caught either at truck start-up or while pulling records from DynamoDB later in the cycle.

To tackle these problems, we set up automatic backup options. When coordinates are absent, the system uses direct path estimation - while gaps in speed data fall back on a standard truck rate of 80 km/h. Estimated arrival times get recalculated whenever needed based on leftover distance and adjusted velocity figures. Since processing runs non-stop, no records are deleted; rather, reliable placeholder values keep operations stable. Instead of typical heatmap tools for spotting blanks, our model logs such events directly, noting each time defaults activate - giving engineers clear insight into performance.

## Outlier Detection and Treatment

In simulations, outliers in data often come from math mistakes instead of actual real-world anomalies. For instance, OSRM might sometimes suggest unusually long route bends; similarly, ship GPS noise can generate positions beyond normal sea zones. To assess these cases, we tracked total distance patterns per planned path while ensuring truck and vessel movements stayed inside typical limits. If sudden big shifts appeared - like a marine unit jumping over half a degree in lat or lon during one reading - the model adjusted those figures to acceptable levels.

Due to outliers affecting plot precision along with model input quality, adjustments are made instead of deletions. During building stages, evaluations resembling box plots confirmed total distance trends rose steadily, while step-by-step gaps stayed inside plausible limits. Such built-in validations stop irregular points from entering outputs meant for display or analysis.

## Data Type Correction and Formatting

A large part of prep work focused on keeping data types uniform throughout the system. Because DynamoDB needs numbers as Decimal objects instead of Python floats, every value for latitude, longitude, distance, or speed was carefully transformed using conditional checks prior to entry. Timestamps like `started_at` or `eta` were adjusted into ISO 8601 format through dedicated formatting rules so different parts of the application - backend, frontend, even third-party tools - could reliably use them.

Geospatial data needed precise type adjustments: since OSRM delivers coordinates as `[lon, lat]`, these were restructured into `(lat, lon)` pairs. Instead of keeping route outputs as dictionary lists, they became standard Python tuples once simulations updated. Consistent format use ensures number-sensitive tasks - like estimating arrival times, gauging leftover distances, or creating machine learning features - run without errors.

## Normalization and Scaling

Even artificial data works better with normalization, particularly in machine learning contexts. Prior to integrating ML predictions into the simulation process, we checked how main inputs - like intended speed, leftover distance, or total advancement - were spread out. Leftover distance, for instance, tends to cluster low early on, then shifts toward higher values nearing destination. So that methods such as K-Means grouping and logistic prediction would run smoothly, we adjusted feature scales via z-scores while training models.

Despite small runtime adjustments in the simulation to reduce extra load, checks ensure inputs stay within normal number limits. During training, visual analysis of data before and after scaling showed extreme values didn't skew results - either for grouping or labeling patterns.



## Subsetting the Data

For analysis and modeling, not all simulated fields were necessary. We subset the data to focus on features most relevant to our machine learning objectives:

- planned\_speed\_kmh (continuous)
- dist\_remaining (continuous)
- stop\_count (engineered categorical, currently zero in the simulation)
- arrival\_status (binary classification label for supervised training)

By filtering down to these variables, we reduced dimensionality and ensured that the models learned from data directly tied to operational performance. Additional fields such as raw coordinates, route polyline data, and cumulative distances were retained only for visualization and geospatial logic. This separation allowed the ML models to operate efficiently without being influenced by large, high-granularity geospatial structures.

By filtering down to these variables, we reduced dimensionality while keeping inputs closely linked to real-world operations. Aside from those, details like GPS points, path lines, or total distance stayed in use - but just for maps and location-based functions. Thanks to this split, machine learning tools ran faster, avoiding interference from dense spatial datasets.

The subsetting process preserves a clean distinction between:

- High-frequency spatial data (used for the map)
- Low-dimensional operational data (used for ML)

This distinction helps keep the ML component lightweight and interpretable while maintaining full simulation fidelity.

---

## 4.5 technical-details/supervised-learning.qmd

“‘markdown

title: “Supervised Learning”

### Supervised Learning

#### Introduction to Supervised Learning in the Simulation System

Supervised learning drives predictions in our logistics simulation. Although clustering uncovers hidden patterns and phases across truck activities, prediction relies on past data to estimate what might happen next. Real-world logistics uses forecasting to spot delays, identify problems early, improve resource use, or act ahead of disruptions. To mirror this behavior, the system applies a classification method that judges how likely a truck is to face major delays given its current status.

Introducing a guided prediction feature shifts the simulation beyond basic visual output, turning it into a practical aid for choices - showing how algorithms boost supply planning using modeled information instead of real inputs.

#### Defining the Prediction Task: Delay Risk Classification

The supervised learning task focuses on predicting delay risk, defined as the likelihood that a truck will arrive substantially later than its OSRM-estimated arrival time.

Target Variable (Label) We constructed a binary target variable, `is_delayed`, where:

- 0 = Truck arrived on time or early
- 1 = Truck arrived late beyond a threshold (e.g., 15% over OSRM ETA)

Because real-world delay labels do not exist in a simulation, we generated labeled data by running multiple simulated journeys and computing actual arrival times versus expected arrival times. This mirrors how organizations bootstrap internal prediction systems without full historical records.

## Training Dataset Construction

To train the supervised model, we generated a dataset consisting of simulated truck journeys running across multiple city pairs with randomized offsets, different OSRM route geometries, and varying average speeds. For each truck, the following features were captured at the beginning of the journey:

| Input Features | Feature                 | Description                                               | Why It Matters                                                                            |
|----------------|-------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------|
|                | Planned Speed (km/h)    | Estimated from OSRM travel time and distance              | Lower planned speeds correlate with difficult routes such as mountainous or urban regions |
|                | Distance Remaining (km) | Total kilometers left in the journey at prediction time   | Longer routes naturally carry higher uncertainty and increased delay probability          |
|                | Stops Feature           | Count of simulated mid-route stops (set to 0 in baseline) | Placeholder for future enhancements such as port bottlenecks or roadblocks                |

## Model Choice: Logistic Regression

We selected Logistic Regression as the primary classifier for delay prediction. The model offers several advantages suitable for simulation-based predictive analytics:

- Highly interpretable coefficients
- Efficient to train and very lightweight, enabling microsecond-level inference
- Robust baseline for probabilistic classification
- Well-suited to binary delay vs. non-delay outcomes

Although more complex algorithms (e.g., Random Forests, Gradient Boosting) could offer marginal accuracy improvements, Logistic Regression aligns best with our goals of transparency, speed, and ease of integration with the backend.

## Training Procedure

1. Data generation Multiple runs of the simulation produced a training corpus of journeys with real arrival times and predicted arrival times.
2. Feature engineering Numerical features were standardized, and the target variable was computed.
3. Train/test split 80% of examples were used for training; 20% for testing.
4. Model training Logistic Regression was fit using scikit-learn:

```
model = LogisticRegression() model.fit(X_train, y_train)
```

5. Evaluation metrics We evaluated performance using:

- Accuracy
- Precision & Recall
- ROC-AUC
- Confusion Matrix

These metrics quantify how well the model distinguishes between on-time vs. delayed trips.

## Model Deployment and Integration

After training, the supervised model was serialized using joblib and deployed inside the backend server's /models directory:

`delay_model = joblib.load("models/delay_model.pkl")` During each simulation update cycle, the backend computes real-time features:

- `plan_speed_kmh`
- `dist_remaining`
- `stops_feature` (currently 0)

Then it performs inference:

```
proba = delay_model.predict_proba([[plan_speed_kmh, dist_remaining, stops_feature]])[0]
delay_risk = float(proba[1])
```

This `delayRisk` value is added to the WebSocket payload alongside cluster ID and real-time positions.

## Real-Time Application in the Live Simulation

The supervised model allows the logistics map to display dynamic, predictive insights for every active truck:

- Each truck marker popup shows "Delay Risk: XX.X%"
- Higher probabilities indicate that the truck is statistically more likely to arrive behind schedule
- Operators can visually scan the fleet for trucks posing elevated risk
- Combined with cluster assignment, the system provides both descriptive (cluster) and predictive (risk) intelligence in real time

This mirrors modern predictive logistics systems used by Amazon, DHL, Maersk, and UPS, where ML is embedded directly into operational dashboards.

## **Interpretation and Business Relevance**

Although the supervised model uses simulated labels, the approach reflects widely adopted practices in supply-chain intelligence:

- Predictive delay modeling helps allocate resources proactively
- Early warning allows for dynamic rerouting or dispatching backup vehicles
- Risk predictions improve reliability and customer satisfaction
- Provides a scalable foundation for adding more sophisticated ML models later

In our project, the model not only enhances realism but also demonstrates how machine learning integrates into real-time operational decision systems.

## **Limitations and Future Extensions**

- Because the training labels come from simulations, the model is only as realistic as the assumptions underlying the simulation.
- Additional features (traffic, weather, congestion, road type) would improve predictive fidelity.
- The current stops\_feature is a placeholder; implementing real bottlenecks would produce richer patterns.
- More advanced models (Random Forests, Gradient Boosted Trees, Neural Nets) could capture nonlinear relationships.

Despite these limitations, the current system provides a robust foundation that illustrates the value of supervised analytics in a logistics context.

## **Conclusion**

The supervised learning component transforms the project from a simple visualization of moving trucks into an intelligent system capable of forecasting operational risks. By predicting delay probabilities in real time, the system showcases how machine learning can meaningfully enhance situational awareness and decision-making in logistics—whether in a simulated environment or a real-world supply chain.

---

## 4.4 technical-details/unsupervised-learning.qmd

“‘markdown

title: “Unsupervised Learning”

# Unsupervised Learning

## Introduction to Unsupervised Learning in the Logistics Simulation

Unsupervised learning helps spot new behavior trends in our simulated logistics setup - using live data instead of pre-labeled results. As the simulation produces constant updates like vehicle speed, location, distance covered, or task completion rates, this method reveals hidden organization in fast-moving streams of detailed information. It’s key for noticing when trips differ from norms, recognizing unusual motion sequences, while grouping vehicles based on how they perform. In our work, it supports ongoing tracking of fleets and improves later predictive systems by adding subtle but useful data cues.

## Clustering Objective and Rationale

The goal here is grouping trucks by how they operate throughout a trip. Although every truck moves along the same set path between start and end points, their behaviors aren’t identical - expected speeds change according to OSRM time predictions, left-over distances shift unevenly due to differing departure times, while outside factors like rerouting or minor delays add further differences.

Clustering these operational characteristics helps us answer several important logistical questions:

- Which trucks are progressing unusually slowly or quickly?
- Are there natural groupings of vehicles based on route difficulty or distance remaining?
- Can we detect potential anomalies early by comparing a truck’s behavior to its cluster peers?
- How do operational factors naturally segment across different routes?

Unsupervised segmentation provides actionable insights even in a simulated setting, aligning closely with industry fleet-monitoring practices.

## Feature Selection for Clustering

We cluster each truck using two continuous operational features, chosen for interpretability and real-time relevance:

1. Planned Speed (km/h) Derived from OSRM's estimated travel duration, this value predicts how fast a truck should move over the entire route. Differences arise from route geometry, road quality, and length.
2. Remaining Distance (km) Calculated as:

$\text{remaining} = \text{total\_distance\_km} - \text{cumulative\_distance\_traveled}$  This feature indicates where the truck is in its journey and distinguishes early-route trucks from those nearing completion.

Together, these features capture:

- Expected vs. actual performance potential
- Progress dynamics
- Proximity to destination
- Route-specific difficulty

They also produce well-separated clusters that are meaningful in real operations.

## Clustering Methodology: K-Means

We employ K-Means clustering, a widely used algorithm for partitioning numerical data into  $k$  distinct groups based on similarity.

Training procedure: A dataset of simulated truck journeys was collected across multiple routes.

For each logged timestamp, `planned_speed_kmh` and `dist_remaining_km` were extracted.

The data was normalized (if necessary) to stabilize cluster boundaries.

K-Means was fit with  $k = 3$ , producing the following conceptual clusters:

| Cluster                           | Interpretation                          | Behavioral Pattern                                  |
|-----------------------------------|-----------------------------------------|-----------------------------------------------------|
| 0 – Long-Distance / Early Journey | High remaining distance, moderate speed | Truck has just begun its route                      |
| 1 – Mid-Route / Steady State      | Moderate distance remaining             | Most trucks spend the longest time here             |
| 2 – Arrival Phase / Low Distance  | Nearly complete                         | Truck is close to the depot and progress stabilizes |

## Real-Time Application in the Simulation

Once deployed, the trained model is loaded by the backend and applied to each truck every simulation tick:

```
cluster = kmeans_model.predict([[plan_speed_kmh, dist_remaining]])[0]
```

The assigned cluster is then included in the WebSocket payload and displayed on the frontend map inside each truck's information popup.

This enables: Live monitoring of fleet behavior

- Pattern detection (e.g., which clusters appear most often)
- Quick visual segmentation of trucks along the route
- Future anomaly detection (trucks behaving differently from cluster peers)

Even though this is a simulated environment, the clustering system behaves exactly like one used in real-world fleet analytics.

## Insights Gained from Clustering

During EDA and deployment, clustering revealed several useful insights:

- Trucks with longer or more complex OSRM routes tend to fall into higher-distance clusters early on.
- Variability in planned speed often creates distinct cluster boundaries even when trucks are the same distance from the destination.
- The cluster IDs provide a simple but effective indicator of operational phase: beginning, middle, or near completion.
- These cluster features became important inputs to subsequent supervised models (e.g., predicting delay risk).

## Limitations and Considerations

While K-Means is simple and effective, several limitations need to be acknowledged:

- Clusters are spherical and may not capture nonlinear route behaviors.
- Synthetic data reduces variability, which can make clusters appear cleaner than in reality.
- The model must be retrained if new features are introduced or if simulation behavior changes drastically.



- Scaling issues may arise if the feature magnitudes differ significantly.

In future extensions, more advanced unsupervised methods—such as DBSCAN for anomaly detection or Gaussian Mixture Models for probabilistic segmentation—could enrich system insights.

## **Conclusion**

Using unsupervised methods helped uncover hidden patterns in the simulated delivery data. Because trucks were grouped by target speed and left-over distance, insights became clearer during live operation. Such grouping links actual simulation output with later prediction models that rely on labeled examples.